

What are ports?

Ports are numbered communication endpoints that allow multiple network services to operate simultaneously on a single server by directing traffic to specific applications. So inside a server there are multiple ports, specifically 2^{16} ports or 65536 ports. They range from 0 to 65535.

To put it simply, ports are like ~~doors~~ virtual doors that allow data exchange between the two sides. To Recap, the IP address says which ~~server~~ server the data should go to, and the port number is which application the data should go to inside that server.

Port numbers that are preassigned ~~and~~ ~~fixed~~ to a specific task/application exist. eg.
(80/443) → web browser. (25/110) → email client.
(21) → file ~~applications transfer~~ transfer application.

Real Example:

When you type: "https://www.yourwebsite.com"

what happens is as follows:

- ① The browser establishes a secure connection (https).
- ② The browser initiates a Transmission control protocol (TCP) connection to the server's IP address, specifically on port 443 (The default HTTPS port).
- ③ Your browser and the server perform an SSL/TLS handshake to establish encryption.
- ④ Finally, the server ~~that~~ sends the webpage data back to your browser through the same port 443.

Where does your operating system role?

Your operating system enables you to have a port 443 that can receive webpage data then display it on a browser.

What is a Domain Name System? (DNS)

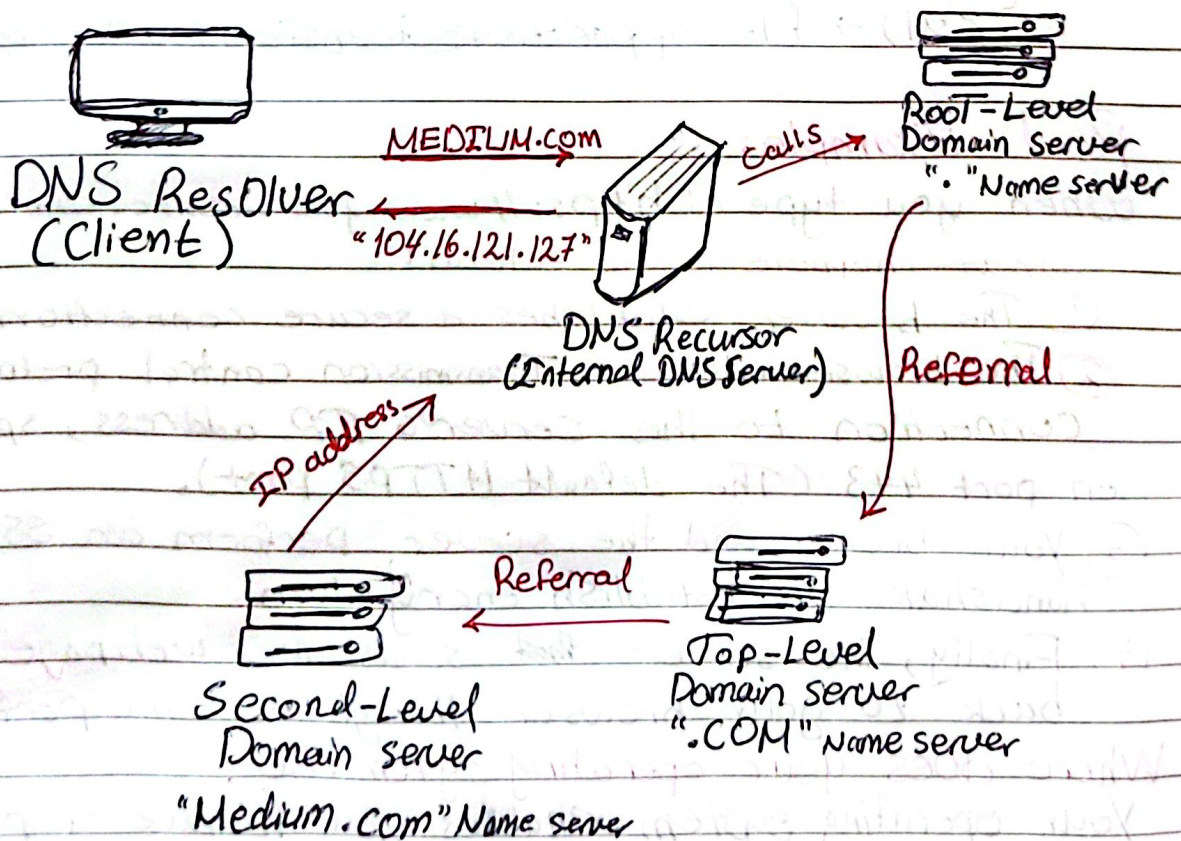
- * It maps human readable domain names to a unique network address (IP)
- * There is a TLD ^{server} for each top-level domain, like .com, .org, .net

Types of DNS servers:

- ① Recursive resolver
- ② Root Nameserver
- ③ TLD Nameserver
- ④ Authoritative Nameservers

- * A DNS server holds the actual ~~Records~~ DNS records like IP addresses for specific domains.

The DNS Recursor:



Fetch API in JavaScript

Fetch is an API used in JS to load Resources in a webpage. It can read files (text, JSON, etc.).

The Fetch Function (`fetch(path)`) returns a promise

Promises

A promise is an object used to manage asynchronous results. Has a `then()` method that lets you execute some functions on Success or on Error. It allows you to build a chain of Asynchronous results.

[ex.] How is a Fetch API used?

```
Fetch("file path").then(onSuccess, onFail);
```

↗ same as

```
const promise = Fetch("file path");  
promise.then(onSuccess, onFail);
```

A promise is in one of 3 states:

- Pending
- Fulfilled
- rejected

* The success function for Fetch gets passed a "response" parameter that contains info about the promise. "response.status" returns the status.

* The Error function for Fetch gets passed a "error" parameter that holds info about the promise.

~~The Promise object has a `text()` method. `text()` returns a promise.~~

you can call the method "`text()`" on a response to convert the response to a readable format. The `text()` method returns a promise object.

Generally, the `then` method receives the value which the promise was fulfilled with. More precisely, the value gets passed to the **PARAMETER** of the ~~first~~ callback function passed to the `then` method.

SO, when a promise is fulfilled, the `then` method is called and the value passed to `resolve` (in the promise declaration) will be passed to the parameter of the callback function passed to `then`.

Inside the callback function passed to `then`, if the callback function returns a normal value (not a promise object), then the `then` will return a **NEW** promise that resolves with the value you returned in the callback function. If you return a promise object in the callback function, then the `then` method waits for that promise and then ~~creates~~ a ~~new~~ ~~promise~~ returns a **NEW** promise object that ~~is~~ ~~it~~ adopts ~~that~~ the resolve value of that promise. All in all, the "`then`" always returns a ~~new~~ **NEW** promise object.

When a promise is fulfilled, the resolve value **ALWAYS** gets passed as a parameter to the callback function passed to `then`.

Limitations of Fetch

you cannot fetch a resource that is hosted on `file://` (file protocol)

What is an API Endpoint?

It is simply a base URL Developers can use to use ~~perform~~ REST API methods (e.g. GET, POST, DELETE)

Example:

Base URL: `https://api.spotify.com`

HTTP method: GET

API Endpoint: `https://api.spotify.com/v1/albums/<id>`

Returns: JSON data about that album request



The fetch method issues a HTTP GET request by default.

So, if you do:

```
fetch("https://api.spotify.com/v1/albums/7DA67S...")  
  .then(onResponse).then(onTextReady);
```

you will receive and load the JSON data as a Javascript object.

CORS: Cross-Origin Resource Sharing.

It is basically browser policies for what resources a webpage can load. A policy that allows or ~~rejects~~ resources from being loaded in a web page.

CORS works when there are requests that are cross-origin, meaning those requests are to another domain which is different than the domain of the current web page.

Same-origin requests are always allowed. So your website can freely fetch from its own domain.

Cross-origin Requests can either be blocked or accepted.

→ Always Allowed:

CSS, images, and JavaScript Files

→ ~~Always~~ Blocked by default:

when `fetch()` or `XMLHttpRequest` is used to request data from another Domain, the browser blocks it unless the server explicitly allows it with CORS headers. This is a security measure to prevent malicious websites from stealing your data.

Single-threaded Asynchrony

The problem we ~~are~~ are trying to solve: what if in our code we try to do an operation ~~on~~ or call a method on a variable that should be assigned a value after the fetch completes? If the connection isn't good, the fetch will work slowly, and any variable expecting a value ~~using~~ from the fetch will still be undefined. This will cause errors however and problems. How do we fix that?

The problem really is we have written our code in a way that assumes fetch will complete before clicking, but with slow connection, that's not a safe ~~ass~~ assumption.

Solutions to this?

- ① Disable buttons until JSON loads
- OR ② Don't show buttons until json loads
- OR ③ Don't show entire LTI until the JSON loads.
- OR ④ You can initialize the variable that is dependant on fetch so that applying methods on it won't bring any errors.

Single-threaded Asynchrony (Cont.)

Is it possible that an event callback function be called (That has to do with the variable that depends on the fetch) while fetching is happening? Meaning is it possible that both ~~events~~ actions will happen at the same time?

Drumroll Please...

The answer is NO, why? because JavaScript is single threaded.

Single-threaded: When your computer processes one command at a time. There is only one call stack.

Multi-threaded: When your computer processes multiple commands simultaneously. There is one call stack per thread. (There are many threads)

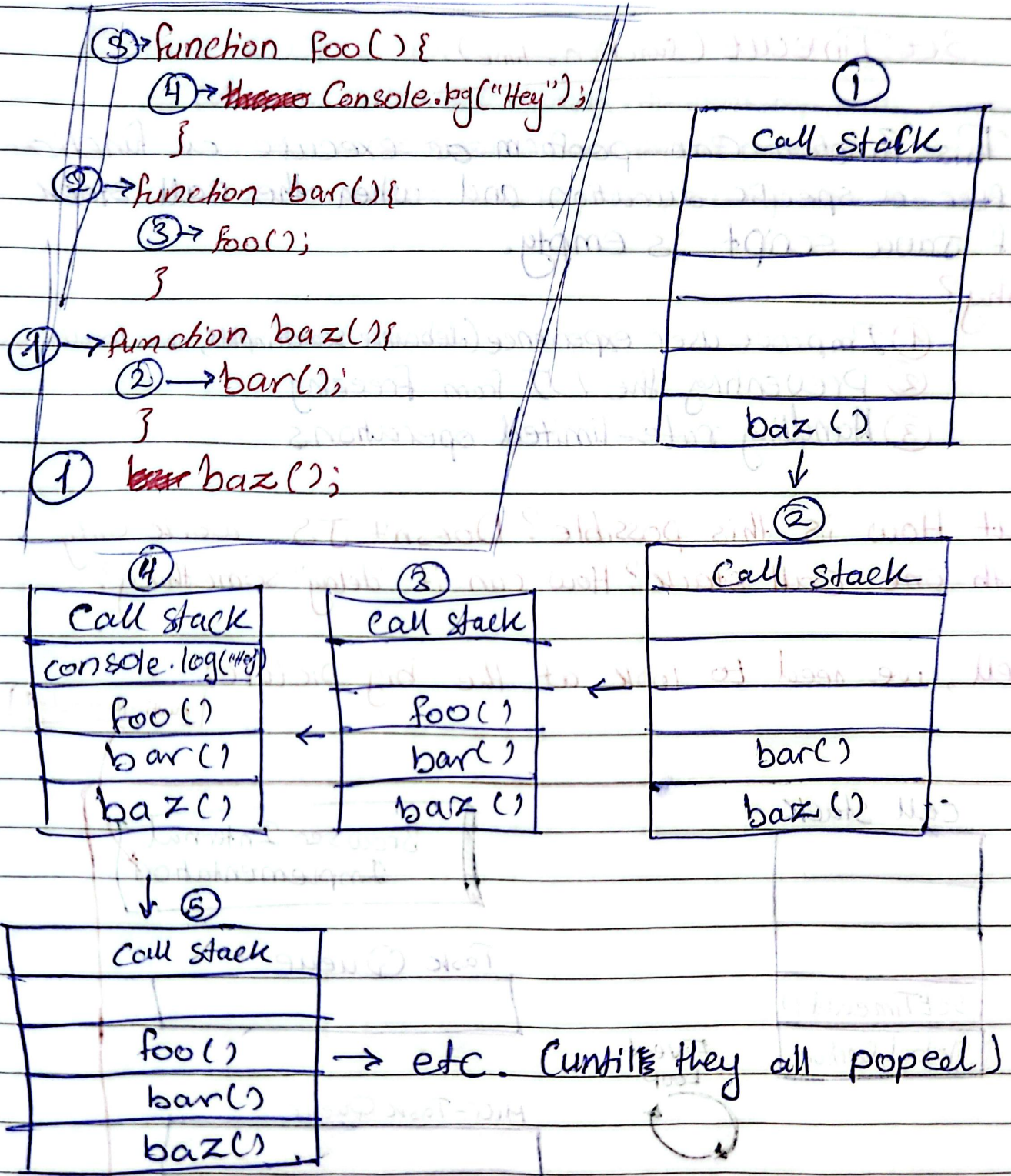
Thread: A linear sequence of instructions. Basically an executable container for instructions

What the hell is a call stack?!

It is basically a stack (The data structure) that stores (or pushes) whatever function we are executing now and pops whatever function we are at now that returns something (finishes)

Note: One Thread = one call stack = one thing at a time

Visualization of call stack



Set Timeout (function, time)

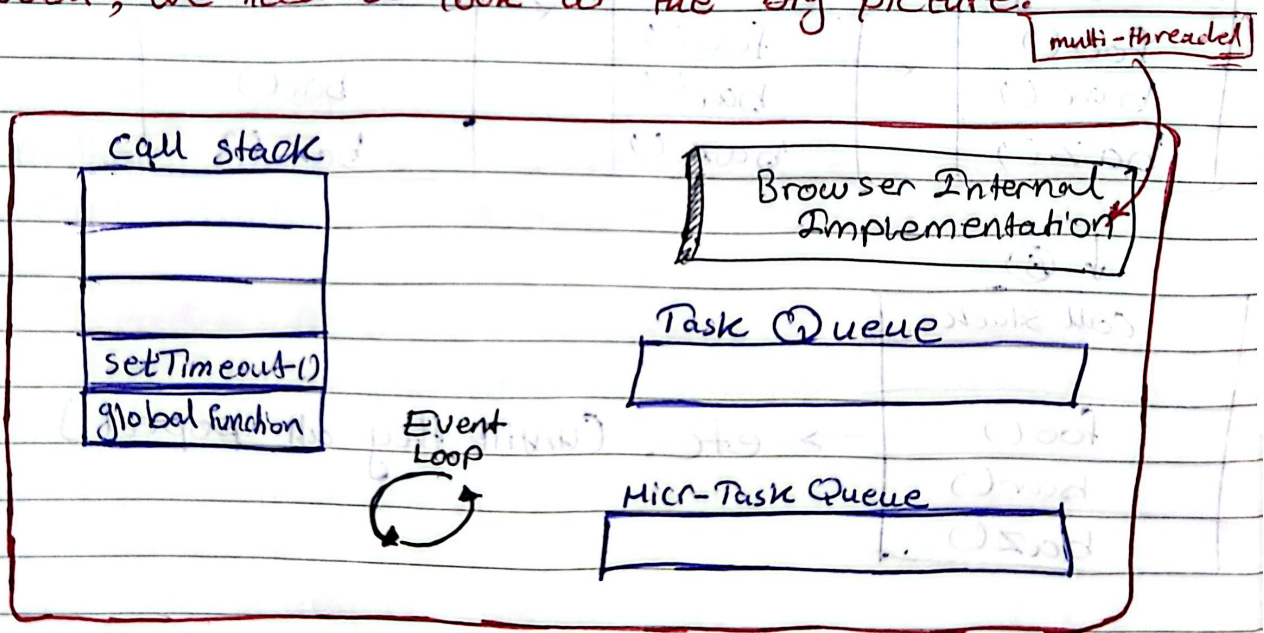
This function can perform or execute a function after a specific duration and when the call stack of JavaScript is empty.

why?

- ① Improve user experience (debounce search inputs, show loading, ...)
- ② Preventing the UI from freezing
- ③ Handling rate-limited operations

But How is this possible? Doesn't JS work only with one call stack? How can it "delay" something?

Well, we need to look at the big picture:



Call stack: JavaScript runtime call stack. Executes the JavaScript commands, functions.

Browser internal implementation: The C++ code that executes in response to native JavaScript commands, e.g. `setTimeout`, `element.classList.add("style")`, etc.

NOTE: The Browser itself is multi-threaded

Micro-task Queue: Promises are special tasks that execute with higher priority, so they have their own special queue.

Event Loop: processes the task queues.

- When the call stack is empty, the event loop pulls the next task from the task queues and puts it on the call stack.
- The micro-task has higher priority than the Task Queue

Servers

- The programming we have been doing to for is called "Client-side" programming.
- The code we write get executed in the ^(client) user's browser.

We will now learn about server-side programming

- The code we write gets run on a server.
- Servers are computers that run programs to generate webpage and other web resources.

Let's have a closer look...

Once you type a link (or URL) to a specific web page, ~~you are asking~~ The browser sends a HTTP GET request saying "Please get ~~to me~~ the index.html file at `http://cs193x.stanford.edu`"

→ Let's have a closer look at this...

The Browser's C++ code creates an array of bytes that is formatted in using HTTP request message format. Then, once the array is ready, the browser asks the operating system, "Hey! Can you send this HTTP GET request message to `https://cs193.stanford.edu`"?

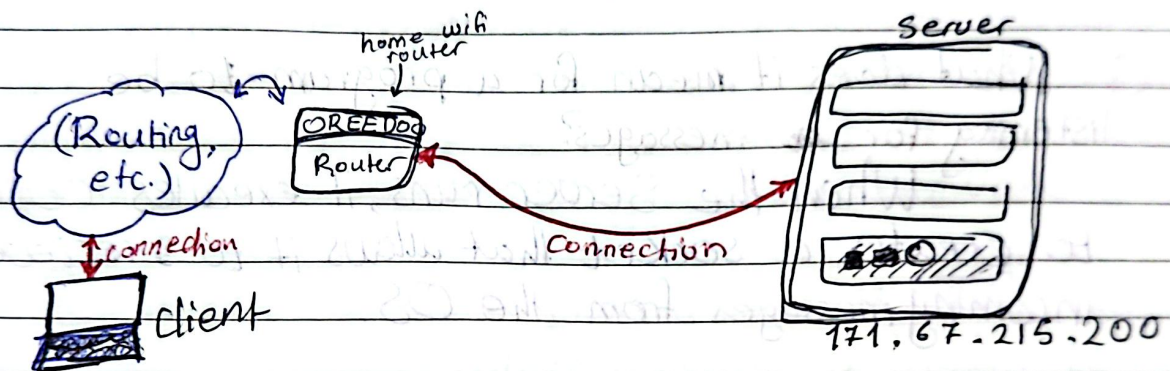
The operating system sends a DNS query to look up the IP address of `http://cs193.stanford.edu`. Then, the DNS server replies with the IP address eg. 171.62.536.200

DNS: Domain Name system, Translate domain names to IP address of the computer associated with that address.

IP address: Numerical unique identifier for every computer connected to the internet.

Then, The operating system opens a TCP connection with the computer at 171.67.215.200. After the TCP connection is established, the OS can send the HTTP message to 171.67.215.200 through the TCP connection.

TCP: Transmission Control protocol, defines the data format for sending information over the wire. Can be used for HTTP, FTP, etc.



Server: There is a computer that is connected to the internet at IP address 171.67.215.200

On this computer is a web server program:

→ The web server program is listening for Incoming messages that are sent to it.

→ The web server program can respond to messages that are sent to it.

AKA NodeJS

↓
Node: The platform we will use to create a web server program that will receive & respond to HTTP Requests.

Side Note What is the definition of a server?

→ Sometimes "server" ~~refers to~~ means the machine/computer that runs the server software.

→ Sometimes "server" means the ~~the~~ software program running on the machine/computer.

you should try to understand the context to know which of them is meant.

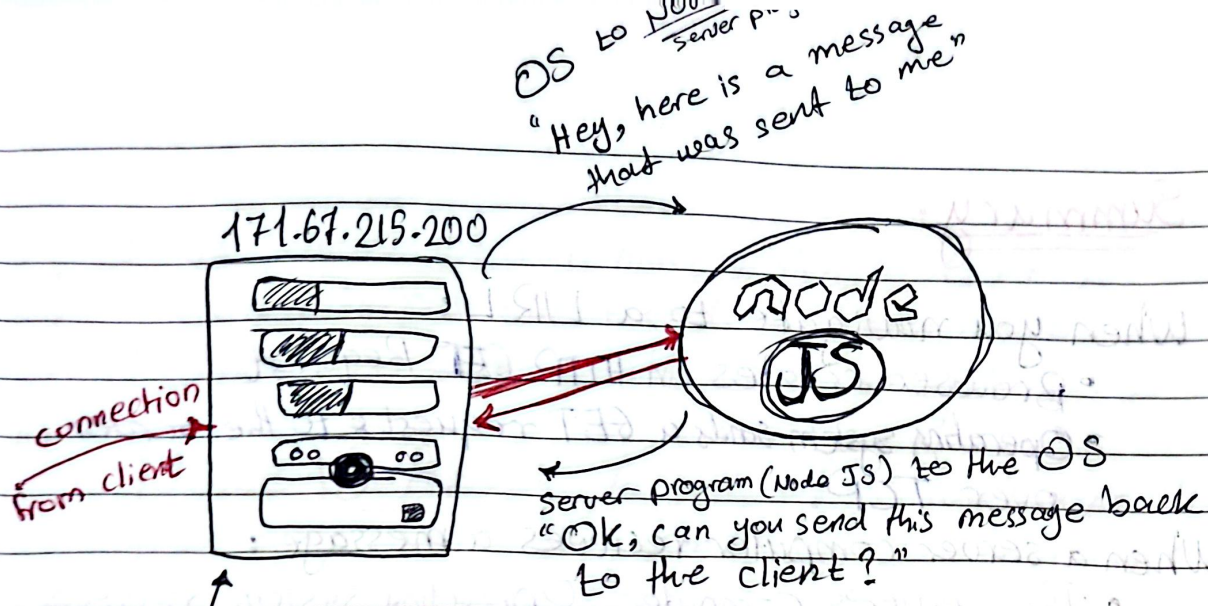
Q: What does it mean for a program to be listening for ~~a~~ messages?

When the Server runs, it executes code to create a socket that allows it to ~~to~~ receive incoming messages from the OS.

Socket: is a one end of a communication channel. You send and receive data on sockets. However, Node JS will abstract this, so you don't need to worry about it (sockets). Because as you know, Node JS enables Javascript to run in the terminal and communicate with ~~the~~ OS.

Finally, A TCP connection is established between the client and the server, so now the client and Server can send messages directly.

What does Node has to do with this?



~~OS to Server~~
The operating system of the server is receiving TCP packets from the wire, and the operating system begins sending the contents of the request to the server ~~again~~ program again (Node JS).

The server software parses the HTTP request and then decides what message it wants to send in response. It formats this message in HTTP, then asks the OS to send this response message over TCP back to the sender.

This HTTP response is then sent back to the client's OS, which notifies the browser of the HTTP response, and then the browser displays the web page.

Summary:

When you navigate to a URL:

- Browser creates an HTTP ~~GET~~ Request
- Operating system sends a GET request to the server over TCP.

When a server computer receives a message:

- The server's computer operating system receives the client's message and sends ~~a message~~ the message to the server software (Node.js) (via a socket)
- The server software (Node.js) then parses the message
- The server software creates an HTTP response
- The server OS sends the HTTP response to the client over TCP

Node JS

- A JavaScript run time written in C++
- Can interpret and execute JavaScript
- Includes support for the NodeJS API

NodeJS API: A set of JavaScript libraries that are useful for creating server programs

V8 (from Chrome): The JavaScript interpreter ("engine") that NodeJS uses to interpret, compile, and execute JavaScript code.

what does this mean?

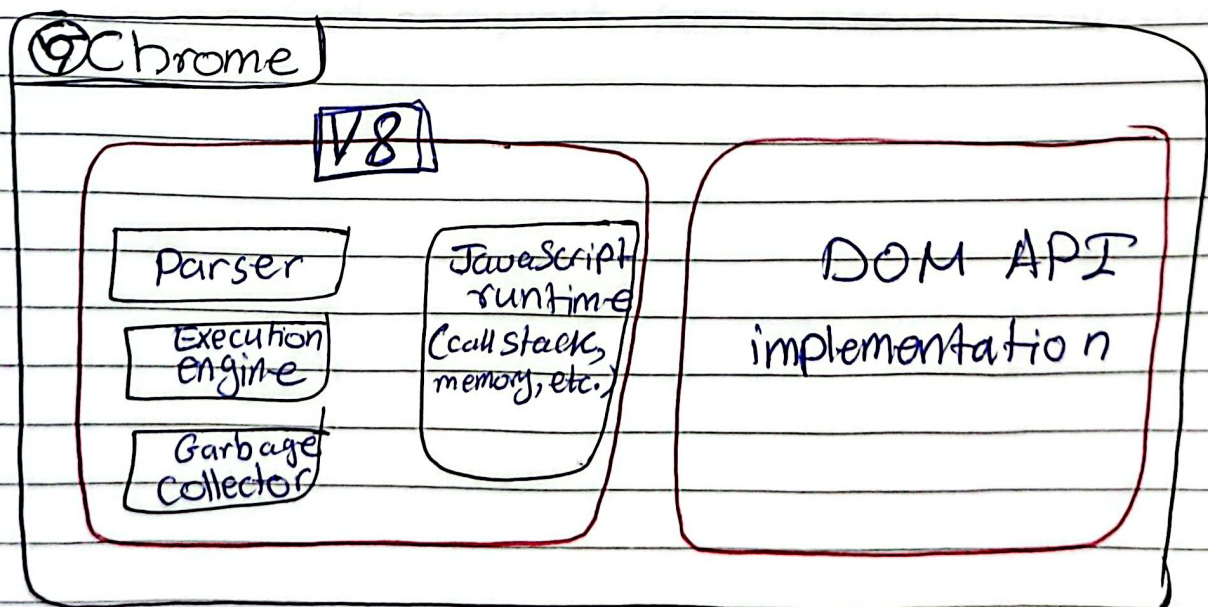
First: Chrome

- A browser written in C++
- Can interpret and execute JavaScript code.
- Includes support for the DOM APIs.

DOM APIs

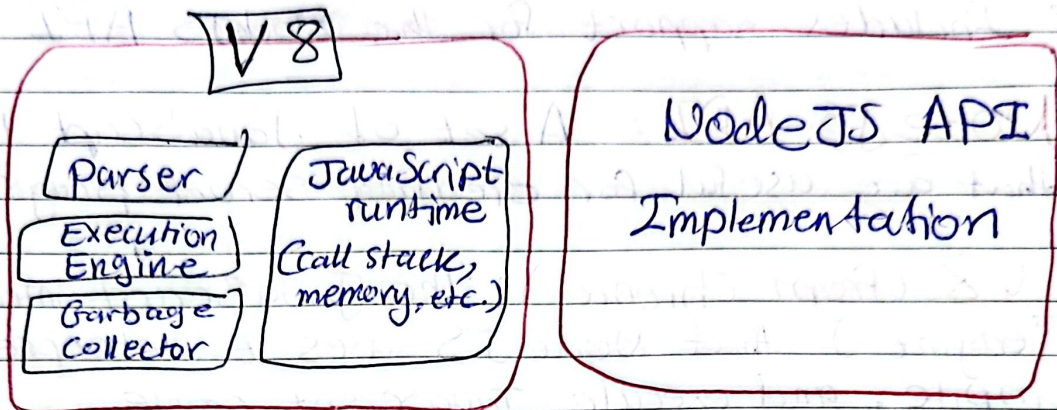
JavaScript libraries to interact with a web page.

V8: "Interpreter" Chrome uses to compile, interpret, execute JavaScript



```
const name = "V8"; console.log("V8");
```


node JS



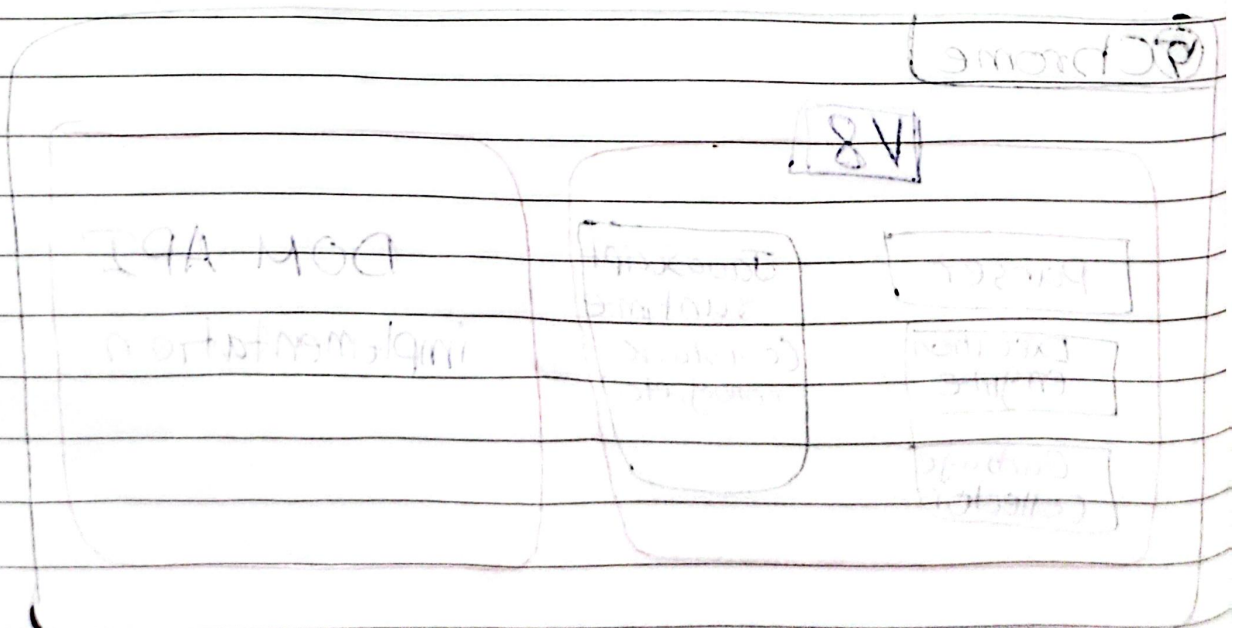
Notice: There is no DOM implementation in Node JS. So something like:

```
document.querySelector("div");
```

in node JS run time will raise an exception.

ReferenceError: document is not defined

To run Node JS, you should install it then you should open a terminal and you will be able to use it. you can also run a JavaScript file with Node JS.



- Using NodeJS enables us to create and manage a server with JavaScript.
- NodeJS is good And Powerful.
- However, when using NodeJS alone, we often have to deal with writing low level instructions a lot.
- So we introduce **ExpressJS** which is a library on top of NodeJS.

Querying our server

There are 3 main ways to send HTTP requests to our server:

- ① Navigate to a URL (e.g. `http://localhost:3000/<path>`) in your browser. This does a GET request only.
- ② Call `fetch()` in webpages.
- ③ `curl` command-line tool. A debug tool (Don't worry about it now)

Curl a command line tool to send and retrieve data from a server.

Syntax:

`curl --request METHOD url`

example `Curl --request PUT http://localhost:3000/hello`

Authentication

Express Session

What is the difference between a session and a cookie?

- A session and a cookie differ ~~in~~ where the data is stored. A cookie has its data stored in the browser. The browser will attach the cookie (the key value pair) to every HTTP request that it does. On the other hand, A session is going to be stored on the server side (the express JS application). The session ~~can~~ stores more data than a cookie.

Note: cookies are attached to HTTP requests.

- A cookie shouldn't be used to save credentials, because a hacker would easily get that information and steal personal data.
- The advantage of a session is that it stores more data and exists on the server side.

Documentation of Express-Session



①

Importing

import session from "express-session"

②

Configure

Once we import express session, we get a function reference. The function is called "session". The function takes an argument "options" to configure the express session. The options is an object that takes keys and values. The properties which options accept are:

Takes

- cookie ~~Object~~ Domain Object
- cookie.domain Date Object
- cookie.expires boolean
- cookie.httpOnly number
- cookie.maxAge boolean
- cookie.partitioned Path Object
- cookie.path String: "low", "medium", "high"
- cookie.priority boolean or String
- cookie.sameSite boolean
- cookie.secure

The session function returns an express middleware that could be used with app.use(.)

Takes

true
false
"lax"
"none"
"strict"

③ Session store implementation

- Deciding what persistent memory we are going to store our sessions in.

A session is used to store information about a particular user moving throughout the browser.

Now the express session middleware to store session data in memory (which is not a scalable solution). We need to set up a ~~memory~~^{session} store.

Meaning we want to connect the middleware with our database. There are many session stores to choose from. We will use connect-mongo.

- So to have this session store working, we should first establish a connection with the database. In our case we would establish connection with MongoDB first. After you create a connection, you should create a store object. We will use MongoStore class from the connect-mongo npm module. We should create a MongoStore object. The constructor of MongoStore takes in an object with some configurations.

```
const sessionStore = new MongoStore({  
  mongooseConnection: mongodb_connection  
  collection: "new collection name"  
});
```

- MongoStore class implements the interface Store, where it specifies the way of storing session data with mongo database.

- After you create a MongoStore object, you assign it to the "store" property in the object "options" that goes to the session factory function of the express-session module.

```
app.use(session({
  secret: "some secret",
  resave: false,
  saveUninitialized: true,
  store: sessionStore,
  cookie: {
    maxAge: 1000 * 60 * 60 * 24
  }
}));
```

- Now once we start listening and run the app, a new session will be created with each ~~an~~ HTTP request. And the session will be saved to mongo DB collection with the collection name you specified before ("new collection name"). Once the session middleware fires with the request, the session ID will be sent via cookie to the client's browser. The cookie is sent in the GET request's (or any request) header. The browser intercepts the request and saves the cookie. The cookie can be viewed via inspect option in the browser. The cookie's default name is connect.sid, but you can change that. The cookie also holds a value that corresponds to the session ID. Now, with that ID of the cookie, ~~the~~ the client can make any request to the server, and the server will

- Check if the id of the cookie of the client corresponds to an existing ID of a session in the DataBase. That way you could do various useful actions like Authentication. So for example, if a GET Request is made, you could check to see if the ID of the cookie in the request corresponds to a session ID, and depending on that you can decide whether to send some content back or not, ensuring Security.

- When you send a cookie to a browser, the client's browser will accept that cookie. And then from that point on, every request made by the client will have the cookie (set-cookie) header attached to it. It is sort of like the browser is like "Hey! there is a cookie that I could use to benefit from"

- The database also will save that session information in a new document.

- Whenever a request is made from the client to the server, the session middleware attaches a "session" property to the request object. The session object holds information about the session that is stored in the DataBase (the session with the same id as the id in the cookie)